

ЯК НАПИСАТИ ВЛАСНИЙ ПЛАНУВАЛЬНИК ЗАДАЧ (SCHEDULER) З НУЛЯ

Цей посібник допоможе вам написати простий планувальник за алгоритмом Round-Robin (по колу), який дозволить вашій ОС виконувати кілька задач "одночасно". Ми використовуватимемо переривання таймера для перемикання контексту.

КРОК 1: КОНЦЕПЦІЯ (ЩО ТАКЕ КОНТЕКСТ І ПЕРЕМИКАННЯ)

Уявіть, що процесор — це людина, яка читає книгу.

1. **Стек (Закладка)** — це місце, де людина зупинилася, коли задзвонив будильник (переривання таймера). У стеку лежать усі її думки (реєстри процесора).
2. **Задача (ТСВ)** — це обкладинка книги. На ній написано номер (ID), і в неї вкладена та сама закладка (вказівник на стек).

Щоб змінити задачу, треба просто забрати у процесора одну книгу і дати іншу, відкриту на її власній закладці. Отже, нам потрібна структура для задачі та логіка збереження закладок.

КРОК 2: ОГОЛОШЕННЯ СТРУКТУР

Створіть файл `task.h` і опишіть цю сутність.

1. **Структура Задачі (`task_t`)** Що має знати про себе задача?
 - **Вказівник на стек (`esp`)**. Зробіть його як звичайне беззнакове 32-бітне число (`uint32_t`). Воно зберігатиме адресу вершини стека.
 - **Ідентифікатор (`id`)**. Унікальний номер задачі (PID).
 - **Вказівник на наступну задачу (`next`)**. Це вказівник на таку саму структуру, щоб зв'язати всі задачі у нескінченне кільце.

КРОК 3: ГЛОБАЛЬНІ ЗМІННІ ТА ІНІЦІАЛІЗАЦІЯ

Створіть файл `task.c`.

1. **Оголосіть змінні:** Створіть глобальний вказівник на поточну задачу (`current_task`), лічильник для ID (почніть з 1) і структуру для "головної задачі" (`main_task`) — це код вашого ядра, який вже працює.
2. **Функція `task_init()`:**
 - Задайте `main_task` ідентифікатор 0, а поле `esp` зробіть 0 (воно збережеться автоматично пізніше).
 - **Головний трюк:** Замкніть кільце. Вкажіть полю `next` структури `main_task` адресу самої ж `main_task` (вона сама собі наступна).
 - Зробіть `current_task` вказівником на `main_task`.

КРОК 4: СТВОРЕННЯ НОВОЇ ЗАДАЧІ

Напишіть функцію `task_create(entry_point)`. Її логіка така:

- Виділення пам'яті:**
 - Викличте `kmalloс` для самої структури задачі.
 - Викличте `kmalloс` для стека задачі (наприклад, 4096 байт). Збережіть це як вказівник на 32-бітне число.
- Підготовка фейкового стека:**
 - Стек росте зверху вниз. Знайдіть кінець виділеної пам'яті (якщо вказівник 32-бітний, додайте до нього 1024). Це ваша вершина стека.
 - Зменште вершину на 1 і запишіть туди число `0x0202` (це прапорець, що дозволяє переривання).
 - Зменште вершину на 1 і запишіть туди `0x08` (ваш селектор коду ядра).
 - Зменште вершину на 1 і запишіть туди аргумент `entry_point` (куди процесор має стрибнути при першому запуску).
- Імітація регістрів:**
 - Зробіть цикл `for` від 0 до 8. У тілі циклу зменшуйте вершину на 1 і записуйте нуль (це пусті регістри загального призначення).
- Заповнення структури і вставка в кільце:**
 - Запишіть отриману фінальну вершину стека у поле `esp` нової задачі. Дайте їй ID і збільште лічильник.
 - Полю `next` нової задачі призначте те, на що зараз вказує `next` поточної задачі.
 - А полю `next` поточної задачі призначте адресу вашої нової задачі.

КРОК 5: ЛОГІКА ПЕРЕМИКАЧА

Напишіть функцію `schedule(current_esp)`. Вона визначає, хто працює далі.

- Перевірка:** Якщо поточна задача не існує (нуль), просто поверніть `current_esp` назад.
- Збереження:** Запишіть переданий аргумент `current_esp` у поле `esp` вашої поточної задачі (ми поклали закладку).
- Зміна задачі:** Зсуньте вказівник поточної задачі на її ж поле `next` (ми взяли нову книгу).
- Повернення:** Поверніть `esp` з оновленої поточної задачі.

КРОК 6: АПАРАТНЕ ПЕРЕМИКАННЯ (АСЕМБЛЕР)

Відкрийте файл `isr.asm` і змініть вашу обгортку таймера (`timer_wrapper`).

- Збережіть усі регістри в поточний стек (використайте інструкцію `pusha`).
- Покладіть поточний регістр стека (`esp`) у сам стек. Це буде аргумент для C-функції.
- Викличте вашу функцію `schedule`. Вона поверне новий `esp` у регістр `eax`.
- Приберіть аргумент зі стека (додайте 4 байти до `esp`).
- Перемикання:** Скопіюйте значення з `eax` в `esp`. З цього моменту процесор вже дивиться у стек нової задачі!
- Відправте сигнал EOI вашому контролеру переривань (запишіть `0x20` у порт `0x20`).
- Відновіть усі регістри (використайте `popa`). Оскільки стек новий, регістри дістануться

вже для нової задачі.

8. Поверніться з переривання (iret).

ЯК ЦИМ КОРИСТУВАТИСЯ НА ПРАКТИЦІ

1. Створіть у C-файлі дві функції з нескінченними циклами (наприклад, task1 друкує "A", task2 друкує "B"). У циклах після друку додайте ще один порожній цикл for на мільйон ітерацій, щоб екран не заповнився миттєво.
2. У kernel_main викличте task_init().
3. Викличте task_create(task1) та task_create(task2).
4. Зробіть у kernel_main свій власний нескінченний цикл, який друкує "-".
5. Коли увімкнуться переривання, на екрані почнуть по черзі з'являтися "A", "B" і "-", тому що таймер перемикатиме керування між цими трьома циклами.

Ось і все! Це повноцінний Round-Robin планувальник задач, який ви можете написати самостійно.